

From lecture 01,

1. What is software?

Ans:

Software is not just a program — it is a combination of three essential components that work together:

Software = Instructions + Data Structures + Documents

1. Instructions (Programs) — Code that tells the computer what to do. Example: The code inside WhatsApp that loads your chats, plays notification sounds, and sends messages.
2. Data Structures — Organized formats that allow programs to store and manage information efficiently. Example: Your contact list stored with name, number, photo — structured so the app can search it instantly.
3. Documents — Manuals, help guides, README files, API docs. Example: Microsoft Word's Help section, or GitHub's README. Without documentation, even great software becomes unusable.

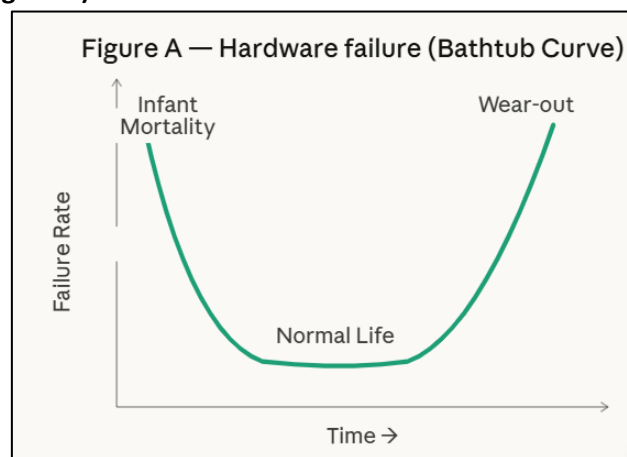
In simple terms, software is the complete package — the code, the data it works with, and the documentation that comes with it. Unlike hardware, software is not manufactured physically; it is developed or engineered using logic, design, and programming. Software cannot be touched or seen, but it controls everything a computer does. It is the "brain" behind all digital operations, from a calculator app to a complex satellite system.

2. Characteristics of software [especially "software does not wear out but it does deteriorate" explanation with those two figures from the slide]

Ans:

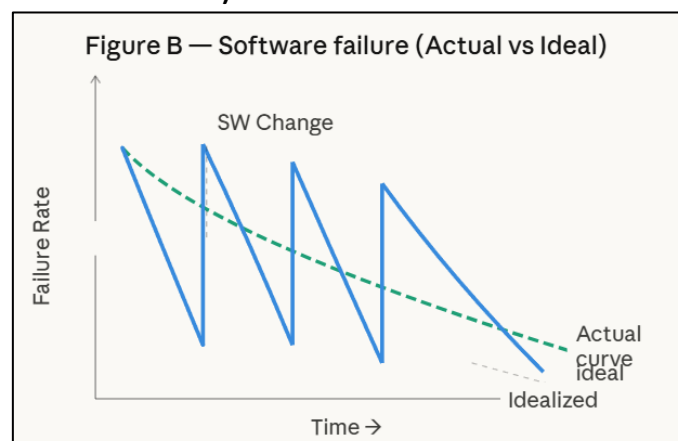
This is one of the most important differences between software and hardware.

Hardware (Bathtub Curve — Figure A):



- Hardware failures are high at the beginning (called "infant mortality") — defects in parts cause early failures.
- Then the failure rate drops to a low, steady level during normal life.
- At the end of its life, the failure rate rises again because of physical wear — dust, vibration, heat damage, etc.
- This creates a U-shape (bathtub curve).

Software (Figure B — Actual vs Idealized Curve):



- In an ideal world, software failure rate should only decrease over time as bugs are fixed (idealized curve — flat green line).
- But in reality, every time software is changed or updated, new bugs (side effects) are introduced — causing sudden spikes in failure rate.
- Over many updates, the overall failure rate gradually rises — this is called "deterioration."
- Software doesn't rust or break physically, but its internal structure becomes messier with every change — this is why it deteriorates.

Key point: Software does not "wear out" like a machine part, but poor maintenance and repeated changes make it harder to manage and more error-prone over time.

3.Applications of software

Ans:

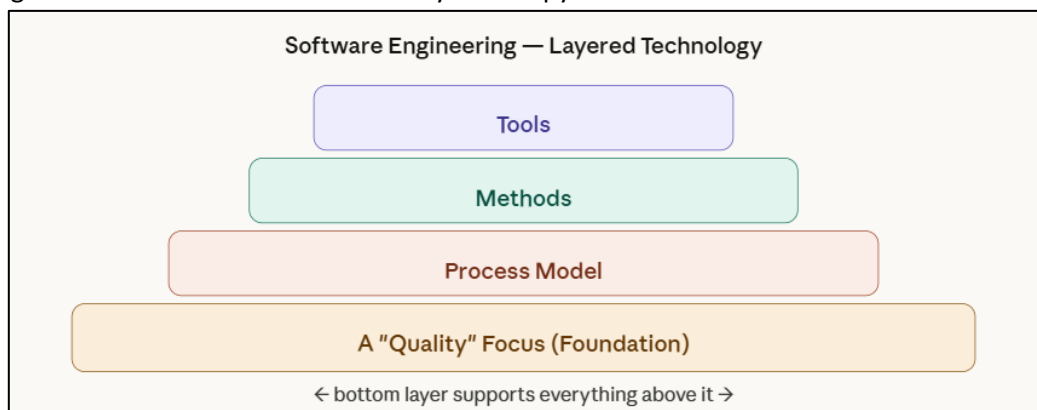
Software can be applied in many different areas. The main types are:

- **System Software** — Programs written to support other programs. Example: Operating systems, compilers, device drivers. It has heavy interaction with hardware.
- **Real-time Software** — Monitors, analyzes, or controls real-world events as they happen. Example: Traffic control systems, flight navigation. It must respond immediately to inputs.
- **Business Software** — Used in organizations for managing data and operations. Example: Payroll, inventory management, accounting systems. This is the largest application area of software.
- **Engineering & Scientific Software** — Uses complex mathematical algorithms ("number crunching"). Example: CAD (Computer-Aided Design), system simulation, weather forecasting.
- **Embedded Software** — Stored in read-only memory (ROM) inside products. Example: Keypad control in a microwave, fuel control in a car's engine. It cannot be easily changed.
- **Personal Computer Software** — Software used by individuals on their own devices. Example: Word processors, spreadsheets, media players.
- **Web-based Software** — Software that runs through a web browser. Example: Websites, web apps using HTML, Java, CGI.
- **AI Software** — Uses non-numerical algorithms to solve complex problems. Example: Pattern recognition, game-playing programs, theorem proving.

From slide 02

4.What is Software Engineering (a layered technology), figure from the first page, description of the layers

Ans: Software engineering is called a "layered technology" because it is built on top of multiple connected layers, each supporting the one above it. Think of it like layers of a pyramid:



- **Layer 1 — Quality Focus (Bottom/Foundation):** This is the base of everything. Any engineering work must be built on a commitment to quality. Techniques like TQM (Total Quality Management) and Six Sigma are used to achieve this.

- **Layer 2 — Process Model:** This is the framework or structure that holds all activities together. It defines what steps to follow, in what order, and what milestones to meet. It enables organized and timely development of software.
- **Layer 3 — Methods:** These are the technical "how-to" techniques used to build software. It includes activities like requirements analysis, design, coding, testing, and maintenance.
- **Layer 4 — Tools (Top):** Software engineering tools provide automated or semi-automated support for the process and methods. Examples: CASE tools, IDEs, testing frameworks.

The bottom layers support the upper layers. Without a quality focus, the process has no reliable foundation. Without a process, methods cannot be organized. Without methods, tools have nothing to automate.

TOOLS (Layer 4 — Top)	Automated/semi-automated support: IDEs, testing frameworks, debuggers, Git
METHODS (Layer 3)	Technical how-to techniques: requirements analysis, design, coding, testing, maintenance
PROCESS MODEL (Layer 2)	Framework holding activities together: Waterfall, Spiral, Incremental, RAD, Prototyping
A 'QUALITY' FOCUS (Layer 1 — Foundation)	Bedrock of everything — organizational commitment via TQM, Six Sigma

5. Descriptions of the phases (definition, development, maintenance etc)

All the models (mainly read in which situation you would choose that model and how it works)

Ans: Generic Phases of Software Engineering + All Process Models

Every software project — regardless of size — passes through three broad phases:

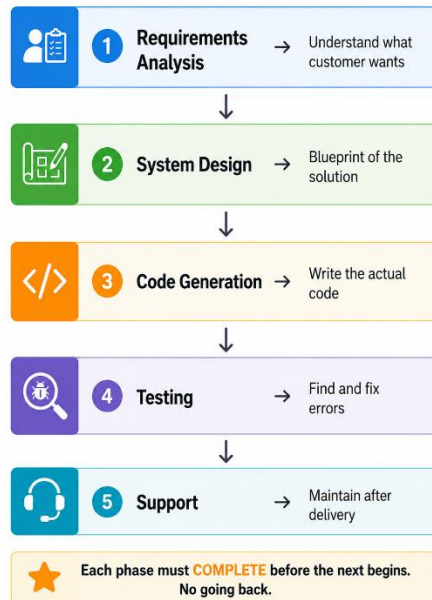


Three Generic Phases:

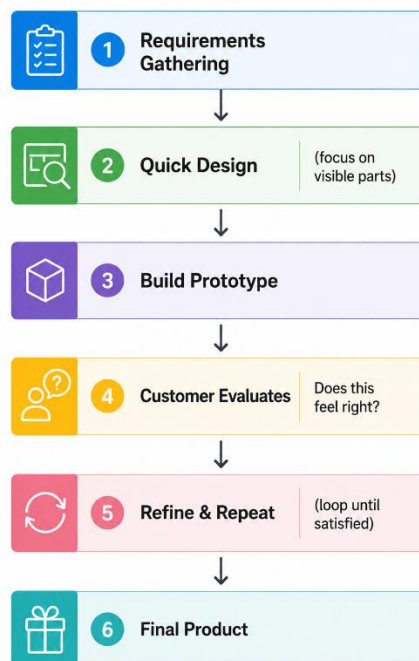
- **Definition Phase** — "What to build?" The team identifies what information is needed, what functions the software must perform, and what the requirements are. Key tasks: system engineering, software project planning, requirements analysis.
- **Development Phase** — "How to build it?" The team decides how data will be structured, how the system will be designed, how code will be written, and how testing will be done. Key tasks: software design, code generation, testing.
- **Support/Maintenance Phase** — "How to maintain it?" After delivery, the software needs changes. Four types: Correction (fix bugs), Adaptation (update for new environment), Enhancement (add new features), Prevention (improve maintainability).

Process Models:

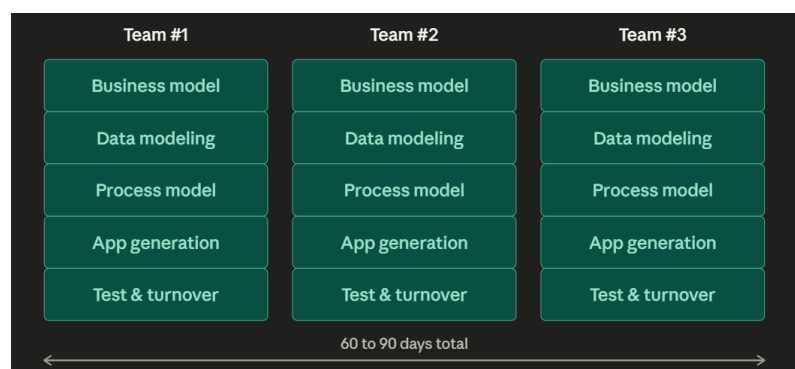
- **Linear Sequential (Waterfall) Model** — Steps follow one by one: Analysis → Design → Code → Test → Support. Best for: well-defined requirements that won't change. Problem: No flexibility; customer waits long before seeing results.



- **Prototyping Model** — A working demo (prototype) is built quickly, shown to the customer, refined based on feedback, repeated until final product. Best for: when requirements are unclear. Problem: Customer may think the prototype is the final product.

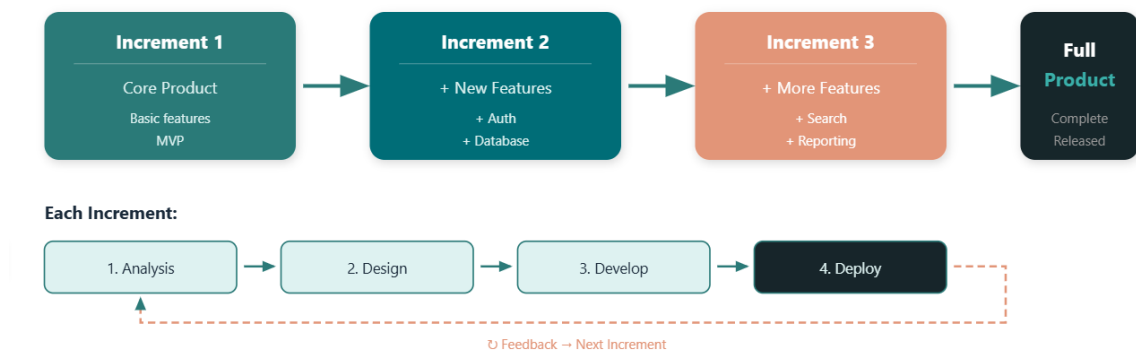


- **RAD Model (Rapid Application Development)** — Multiple teams work in parallel on different parts of the system. Completes a full system in 60–90 days. Phases: Business Modeling → Data Modeling → Process Modeling → Application Generation → Testing & Turnover. Best for: well-understood, modular projects with tight deadlines.

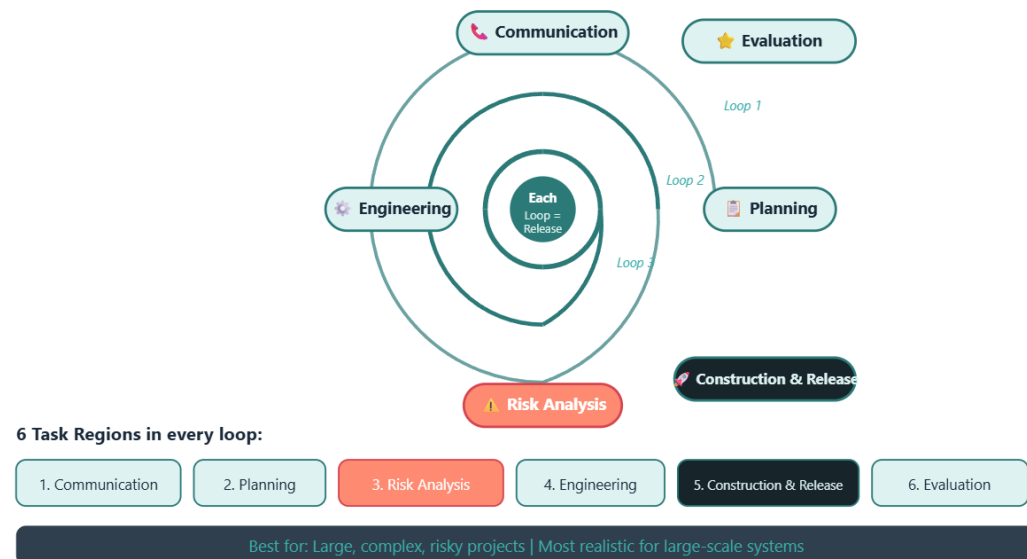


- **Incremental Model** — Delivers the software in pieces (increments). First increment = core product. Each next increment adds more features. Best for: when full staffing is unavailable or requirements evolve.

Incremental Model



- **Spiral Model** — Combines incremental delivery with risk analysis. Goes through loops: Customer Communication → Planning → Risk Analysis → Engineering → Construction & Release → Customer Evaluation. Best for: large, complex, risky projects. Most realistic for large-scale systems.



Quick Reference — Process Models Comparison

Model	Choose When...
Waterfall	Requirements are fully known, fixed, and will not change.
Prototyping	Customer requirements are vague or unclear from the start.
RAD	Project is modular, deadline is tight, enough skilled developers available.
Incremental	Want to deliver core value early and add features gradually.
Spiral	Project is large, complex, high-risk, requirements evolve over time.
Component Assembly	Reusing existing software components is the primary objective.
Concurrent	Different parts of the project are at different lifecycle stages simultaneously.
Formal Methods	Mathematical precision and correctness are critical (aviation, banking).
Cleanroom	Preventing errors before testing is the primary quality strategy.

From Slide 3, (scenario based questions)

Management spectrum :

Effective software project management focuses on four key elements called the Four P's:

People — The most important element. Includes everyone from senior managers to developers, from customers to end-users. The SEI developed a People Management Capability Maturity Model (PM-CMM) to help organizations manage their people effectively. High people management maturity leads to better software engineering practices.

Product — The software to be built. Before development, the team must clearly define the product's objectives and scope, consider alternative solutions, and identify technical/management constraints. Without this, accurate estimation of cost, risk, and schedule is impossible.

Process — The set of framework activities and tasks that guide software development. It provides the structure for planning. Task sets include milestones, work products, and quality assurance points. The right process model must be chosen based on customer needs, product type, and project environment.

Project — The complete effort required to build the product. The project manager is responsible for managing people, product, and process together. Common warning signs of a failing project include: poorly defined scope, unrealistic deadlines, resistant users, changing technology, and loss of sponsorship.

Description of people :

People — Players, Team Leaders, Software Teams, Communication

The Players (5 Categories):

- **Senior Managers** — Define business issues and overall strategy.
- **Project/Technical Managers** — Plan, motivate, organize, and control the team.
- **Practitioners** — Developers/engineers who do the actual technical work.
- **Customers** — Define what the software must do (requirements).
- **End-users** — The actual people who will use the software after release.

Team Leaders — MOI Model: A good team leader needs three qualities:

- **Motivation** — Keep the team energized and focused.
- **Organization** — Structure the team and tasks properly.
- **Ideas** — Encourage innovation and creative problem solving.

Software Team Structures:

- **Democratic Decentralized (DD)** — No permanent leader, decisions by group consensus.
- **Controlled Decentralized (CD)** — Has a defined leader, but decisions are still group-based.
- **Controlled Centralized (CC)** — One leader handle top-level decisions and internal coordination.

Communication approaches: Formal documents, status review meetings, group meetings, email, and interpersonal networking. Good communication prevents most common project failures like unclear requirements and poor coordination.

Coordination and Communication

Most project failures trace back to poor communication — not technical problems. Scale, uncertainty, and interoperability make communication challenging. Five approaches are used:

- Formal, impersonal — Written documents, technical memos, design specifications.
- Formal, interpersonal — Code reviews, design walkthroughs, status review meetings, QA activities.
- Informal, interpersonal — Daily standups, quick team chats, hallway conversations.
- Electronic — Email, Slack, GitHub comments, bug trackers, issue management systems.
- Interpersonal networking — Discussions with outside experts, stakeholders, vendors, and other project teams.

End of Notes — Good Luck on Your Exam!